

Trendly Agent Solution Note

Architecture & Data Flow

The Trendly support agent is architected as an LLM-driven orchestration layer integrated with Next.js API routes, designed to securely interact with simulated backend commerce systems.

Core Components

1. **Next.js Frontend:** A React-based floating widget UI overlaying a mock e-commerce storefront. It manages chat state, session persistence, and renders markdown safely.
2. **Next.js API Route (`app/api/chat/route.ts`):** Acts as the orchestration loop. It manages the session (via cookies), handles conversation history, and invokes the LLM.
3. **LLM Client (`lib/llm/client.ts`):** An OpenRouter-backed client (utilizing models like Gemini or Llama 3) configured to parse tool calls natively via OpenAI-compatible API schemas.
4. **Tool Handlers (`lib/agent/toolHandlers.ts`):** The deterministic execution layer. When the LLM requests data (e.g., `get_order`) or an action (e.g., `initiate_return`), this layer executes business logic against simulated data stores.
5. **System Prompt (`lib/agent/systemPrompt.ts`):** Injects the verbatim Markdown policy document alongside stringent behavioral guidelines and safety protocols.

Execution Flow

When a user sends a message, it is appended to the session history. The orchestration loop invokes the LLM. If the LLM determines it needs external data, it outputs a tool call. The backend executes the specific TypeScript function, appends the JSON result to the history as a tool role message, and re-invokes the LLM. This loop continues (up to a limit) until the LLM returns a final text response.

Key Trade-offs

1. Full-Context Grounding vs. RAG

- *Decision:* The entire 5.8KB Trendly Policy document is injected verbatim into the system prompt rather than using Retrieval-Augmented Generation (RAG).
- *Rationale:* Given the document's small size, injecting it wholesale guarantees 100% recall. RAG introduces retrieval latency and the risk of retrieving irrelevant chunks, which is unacceptable for strict policy enforcement. This scales poorly if the policy grows beyond context windows, but is optimal for current constraints.

2. Deterministic State (Tools) vs. LLM Reasoning

- *Decision:* The LLM is strictly prohibited from determining return eligibility itself. Instead, it must call `check_eligibility` and read the structured JSON result.
- *Rationale:* LLMs struggle with precise date math and complex conditional logic (e.g., "final sale items are exchange only, but shoes without a box have a fee"). Offloading this to deterministic TypeScript code eliminates hallucinations at the cost of requiring more engineering upfront to build robust tools.

3. Stateless APIs vs. Session-Backed Chat

- *Decision:* Conversation history is stored entirely in memory on the server, keyed by a

session ID stored in an HTTP-only cookie.

- *Rationale:* This reduces payload size on the client-side and prevents users from manipulating the conversation history array to bypass verification states. However, in-memory storage means sessions do not persist across server restarts. In a production environment, this would be replaced with Redis.

Known Limitations

1. **Security / Authentication Depth:** The agent verifies identity using an order ID and an email/phone combination. This is a basic form of authentication. If a malicious actor knew an email and an order ID, they could access basic order data. True authentication should tie into a logged-in user session (e.g., JWT or OAuth).
2. **In-Memory Datastore:** The orders and session states are simulated in memory. Concurrent requests from the same user across different servers (in a multi-node deployment) would fail without sticky sessions or a centralized database.
3. **Latency:** Because the agent relies on an LLM loop, complex queries requiring multiple tool calls (e.g., verify -> check_order -> check_eligibility) can result in latency of 3-5 seconds as the LLM is invoked multiple times in sequence.

5 Discovery Questions for the Ops Team

Before deploying this agent in a real production environment, I would ask the Trendly operations and customer experience teams the following:

1. **Escalation Routing:** When the agent escalates a ticket to a human, how are tickets categorized and routed? Do you use Zendesk, Gorgias, or Salesforce? What specific metadata (tags, custom fields) does the agent need to pass via API for seamless handoff?
2. **Refund Authorization Limits:** Should the agent be granted the authority to instantly issue store credit or refunds (e.g., the ₹250 delay credit) without human oversight, or should it draft a refund request for an ops manager to approve? Is there a monetary limit on automated resolutions?
3. **Authentication Strategy:** Are customers required to be logged in to initiate a return, or do you allow guest checkouts? If guest checkout is supported, is Order ID + Email sufficient verification, or do you require an OTP sent to the email for sensitive actions like address changes?
4. **Policy Ambiguity Handling:** What is your preferred protocol when a customer is incredibly frustrated but technically outside the policy bounds (e.g., a VIP customer wanting a return on day 31)? Should the agent maintain a hard line, or should it immediately escalate VIP emails to a specialized retention team?
5. **Data Freshness and APIs:** How real-time are your logistics and warehouse APIs? If an order is marked "delivered" in Shopify but the customer says it isn't there, does the agent trust the API, or does it trigger an automatic carrier investigation workflow?